

ASSESSMENT TOOL 1 — ANNEXURE A: DESIGN TASK

Course Name: Artificial Intelligence **Course Code: CSA1711**

Name : B.BHARGAVI

Reg No : 192311136

Question 1: Hybrid AI Recommendation Engine Design (10 Marks)

1. Problem Understanding & Requirement Analysis

E-commerce and media platforms must recommend items to users at scale, but pure content-based filtering ignores community wisdom while pure collaborative filtering suffers from cold-start for new users and items. The requirement is a hybrid engine that fuses both signals, updates in near real time as users interact, and scales to millions of user-item pairs using distributed processing rather than a single-machine model.

2. System Architecture Design

The system is organised into four layers:

Data Sources → Kafka Ingestion → HDFS/S3 (raw store) → PySpark RDD Processing (Content-Based + Collaborative) → Hybrid Fusion Layer → Recommendation Agent → API / User Interface

- Ingestion Layer — clickstream, ratings, purchase and browsing events land in a message queue (Kafka) and are batched into PySpark.
- Storage Layer — raw interactions in a distributed file store (HDFS/S3); item metadata (genre, tags, price) in a feature store.
- Processing Layer — PySpark RDD/DataFrame jobs compute content similarity and collaborative filtering factors.
- Serving Layer — a hybrid scorer merges both outputs and an intelligent agent layer personalises the final ranking before it reaches the API.

3. Use of PySpark & RDD Operations

- `map()` / `flatMap()` to convert raw logs into (user, item, rating) tuples.
- `groupByKey()` / `reduceByKey()` to build user-item and item-item co-occurrence matrices for collaborative filtering.

- `join()` to combine collaborative filtering scores with content-based similarity scores computed from TF-IDF vectors of item metadata.
- MLlib's ALS (Alternating Least Squares) trained on an RDD of ratings for latent-factor collaborative filtering.
- `cache()/persist()` on frequently reused RDDs (e.g., the item-similarity matrix) to avoid recomputation across iterations.

4. Intelligent Agent Integration

A lightweight recommendation agent sits at the serving layer for each active user session. It observes real-time feedback (clicks, skips, dwell time), adjusts the weighting between the content-based and collaborative scores (exploration vs. exploitation), and triggers incremental model refresh requests when it detects a shift in a user's preference pattern, so recommendations adapt without waiting for the next full batch retrain.

5. Scalability & Distributed Computing Design

- Horizontal scaling by adding Spark worker nodes as the user-item matrix grows.
- Data partitioned by user-id hash so related interactions are co-located, reducing shuffle cost.
- Fault tolerance via RDD lineage — lost partitions are recomputed automatically rather than requiring checkpoints for every stage.
- Batch (nightly full retrain) plus streaming (Spark Structured Streaming for micro-batch updates) processing running side by side.

6. Data Flow & Processing Pipeline

Events flow from source to score in a clearly staged pipeline: raw event capture, cleaning and de-duplication, feature extraction, model scoring, fusion, and agent-driven re-ranking, with each stage logged so a failure in one stage does not corrupt downstream stages.

7. Innovation & Creativity

The design's novelty lies in a feedback-weighted fusion score that lets the agent shift dynamically toward content-based recommendations for new users (cold-start) and toward collaborative filtering as interaction history grows, rather than using a single fixed blending weight for every user.

Question 2: Smart Disaster Detection System (10 Marks)

1. Problem Understanding & Requirement Analysis

Disasters such as floods and earthquakes must be detected within minutes using heterogeneous, high-volume sources — satellite imagery, IoT sensors (seismic, water-level), and social media text — which

vary in format, arrival rate, and reliability. The system must fuse these sources, filter noise, and generate early warnings faster than any single source could alone.

2. System Architecture Design

Satellite / Sensors / Social Media → Kafka Topics → PySpark RDD Cleaning & Normalisation → Data Fusion (per grid cell) → Regional Detection Agents → Central Coordination Agent → Early-Warning Alerts

- Multi-Source Ingestion Layer — separate connectors for satellite feeds, sensor networks, and social media APIs, each pushing into Kafka topics.
- Distributed Processing Layer — PySpark RDD jobs clean, normalise and timestamp-align each stream.
- Data Fusion Layer — combines the three sources into a unified event score per geographic grid cell.
- Multi-Agent Coordination Layer — regional agents monitor their grid cells and escalate to a central alerting agent.
- Alerting Layer — pushes warnings to emergency response dashboards and public notification channels.

3. Use of PySpark & RDD Operations

- `map()` to parse and geotag each incoming record with a grid-cell id.
- `filter()` to discard low-confidence or duplicate sensor readings.
- `reduceByKey()` on grid-cell id to aggregate multi-source signals into one composite anomaly score per cell per time window.
- window-based streaming transformations (Spark Structured Streaming RDD/DataFrame) for continuous 1–5 minute detection windows.
- `broadcast()` variables to distribute static reference data (historical flood/earthquake baselines) efficiently to all worker nodes.

4. Intelligent Agent Integration

Each geographic region is monitored by a dedicated agent that continuously evaluates the fused anomaly score against a locally-tuned threshold. When a regional agent detects a likely event, it communicates with neighbouring regional agents to confirm spatial spread (e.g., a flood moving downstream) before the central coordination agent issues a verified alert, reducing false positives from any single noisy source.

5. Scalability & Distributed Computing Design

- Grid-cell partitioning allows new geographic regions to be added by spinning up additional partitions/agents without redesigning the pipeline.
- Spark's fault-tolerant RDD lineage ensures a failed executor does not lose in-flight disaster detection state.
- Streaming and batch layers run in parallel (lambda architecture) — streaming for immediate alerts, batch for retraining anomaly thresholds.

6. Data Flow & Processing Pipeline

Raw signals are ingested continuously, aligned by timestamp and location, fused into a single composite score per grid cell, evaluated by regional agents, and — on confirmation — escalated through the central agent to the alerting layer, with every stage's output logged for post-event audit and model improvement.

7. Innovation & Creativity

The cross-region agent-confirmation step (neighbouring agents corroborating a detection before alerting) is the key innovation, since it mimics how human disaster-response teams cross-check reports before mobilising, substantially cutting false-alarm rates compared with single-source threshold alerting.

Question 3: AI-Based Fraud Detection Framework (10 Marks)

1. Problem Understanding & Requirement Analysis

Financial institutions process millions of transactions per hour; fraud patterns evolve constantly, and detection must happen within seconds to block a transaction before settlement. The requirement is a scalable, low-latency anomaly detection framework built on distributed processing, with agents that adapt detection rules as fraud tactics change.

2. System Architecture Design

Transaction Stream → Kafka → PySpark RDD Feature Engineering → Distributed Anomaly Scoring → Fraud Detection Agents → Alert / Block Decision → Feedback Loop to Retraining

- Transaction Ingestion Layer — real-time transaction stream captured via Kafka.
- Distributed Feature Engineering Layer — PySpark RDD jobs compute per-account behavioural features (velocity, geo-distance, amount deviation).
- Anomaly Detection Layer — distributed scoring using statistical and ML-based anomaly models (Isolation Forest / clustering on RDDs).
- Fraud Agent Layer — collaborative agents that classify, escalate, and trigger alerts or transaction holds.

- Feedback Layer — confirmed fraud/non-fraud outcomes are fed back to continuously retrain the models.

3. Use of PySpark & RDD Operations

- map() to compute per-transaction derived features (time since last transaction, distance from usual location).
- reduceByKey() keyed by account-id to maintain rolling behavioural aggregates (average spend, transaction frequency).
- filter() to isolate transactions crossing statistical thresholds for deeper agent inspection.
- MLlib's Isolation Forest / K-means clustering trained on historical RDDs to flag outlier transaction patterns.
- persist() on the rolling account-profile RDD since it is read by every incoming transaction in that account's stream.

4. Intelligent Agent Integration

Multiple specialised agents collaborate: a velocity agent watches transaction frequency, a geo-agent watches location jumps, and an amount agent watches spend deviation. Their individual suspicion scores are combined by a coordinating agent that makes the final block/allow/escalate decision, and each agent updates its own thresholds as new confirmed-fraud feedback arrives, allowing the system to adapt to new fraud patterns without a full manual rule rewrite.

5. Scalability & Distributed Computing Design

- Partitioning by account-id keeps each account's transaction history co-located, minimising shuffle during rolling aggregate updates.
- Horizontal scaling of Spark executors to absorb peak transaction volumes (e.g., festive shopping seasons).
- In-memory RDD caching keeps per-account profiles fast to access for millisecond-level scoring decisions.

6. Data Flow & Processing Pipeline

Each transaction flows from ingestion through feature computation, multi-agent scoring, a fused decision, and — for flagged cases — into a review queue whose outcome is looped back as labelled training data, closing the loop between detection and continuous model improvement.

7. Innovation & Creativity

The framework's innovation is treating fraud detection as a multi-agent negotiation rather than one monolithic model — each specialised agent contributes an independent opinion, which is more resilient to a fraudster tuning behaviour to evade a single detection signal.

Question 4: Autonomous Supply Chain Optimization System (10 Marks)

1. Problem Understanding & Requirement Analysis

Modern supply chains span multiple warehouses, suppliers, and transport routes, generating large volumes of inventory, logistics, and demand data. The requirement is a distributed AI system that processes this data at scale and lets autonomous agents make decentralised decisions — reordering stock, rerouting shipments — to minimise cost while maintaining service levels.

2. System Architecture Design

Inventory / Logistics / Demand Data → PySpark RDD Processing (Forecasting) → Warehouse & Route Agents (local decisions) → Coordination Agent (conflict resolution) → Execution (ERP / Logistics Systems)

- Data Layer — inventory levels, order history, and logistics/GPS data streamed continuously into a distributed store.
- Processing Layer — PySpark RDD jobs compute demand forecasts and identify stock/logistics bottlenecks.
- Decentralised Agent Layer — per-warehouse and per-route agents that make local decisions (reorder quantity, route choice).
- Coordination Layer — a higher-level agent resolves conflicts (e.g., two warehouses competing for the same supplier capacity).
- Execution Layer — approved decisions are pushed to ERP/logistics systems.

3. Use of PySpark & RDD Operations

- `map()/flatMap()` to transform raw order and shipment logs into structured (SKU, warehouse, timestamp, quantity) records.
- `reduceByKey()` on SKU-warehouse pairs to compute rolling demand and stock-out risk.
- MLlib regression models trained on historical RDDs for demand forecasting per SKU.
- `join()` to combine forecasted demand with current inventory and incoming shipment RDDs to compute reorder points.
- `sortByKey()` to rank routes/suppliers by cost-efficiency for the agent layer to consume.

4. Intelligent Agent Integration

Each warehouse runs a local agent that decides reorder quantities and timing based on its forecast and current stock, while route agents choose the most cost-efficient logistics path given live traffic and fuel-cost data. Because these agents act on local information, a higher-level coordination agent periodically reconciles their decisions to avoid resource contention (e.g., two warehouses both requesting the same limited supplier capacity), giving the system decentralised speed with centralised consistency.

5. Scalability & Distributed Computing Design

- Each warehouse/route agent operates on its own data partition, so adding a new warehouse only adds a new partition and agent instance.
- PySpark's distributed processing handles the growing volume of transaction and GPS-tracking data without a redesign.
- Asynchronous agent communication (rather than a single global lock-step decision cycle) keeps the system responsive as it scales.

6. Data Flow & Processing Pipeline

Data flows from live inventory and logistics feeds through distributed forecasting, into local agent decisions, through coordination for conflict resolution, and finally into execution systems — with outcomes (actual vs. forecast demand, delivery times) fed back to continuously refine the forecasting models.

7. Innovation & Creativity

The decentralised-decision-with-central-reconciliation pattern is the key innovation: it avoids the latency of a single global optimiser while still preventing the inefficiency of fully independent agents working at cross-purposes.

Question 5: Personalized Learning Analytics Platform (10 Marks)

1. Problem Understanding & Requirement Analysis

An educational platform serving thousands of learners must analyse behavioural and performance data (quiz scores, time-on-task, content engagement) at scale and adapt content delivery per learner in near real time. The requirement is a distributed analytics pipeline paired with agents that personalise the learning path for each student.

2. System Architecture Design

LMS Events (quizzes, videos, forums) → PySpark RDD Analytics (mastery, engagement) → Per-Student Adaptive Agent → Personalised Content Recommendation → Instructor Dashboard / Student Feedback

- Data Collection Layer — learning-management-system events (quiz attempts, video watch time, forum activity) streamed into the platform.
- Distributed Analytics Layer — PySpark RDD jobs compute per-student mastery levels and engagement patterns.
- Adaptive Agent Layer — per-student agents recommend the next content item and difficulty level.
- Feedback & Reporting Layer — dashboards for instructors, and real-time nudges for students.

3. Use of PySpark & RDD Operations

- `map()` to convert raw LMS logs into (student-id, topic, score, time-spent) records.
- `reduceByKey()` on student-id to build rolling mastery scores per topic.
- `groupByKey()` to cluster students with similar learning patterns for cohort-level insights.
- MLlib classification/regression on historical RDDs to predict a student's likelihood of struggling with an upcoming topic.
- `cache()` on the per-student mastery-profile RDD since it is queried repeatedly by the adaptive agent during a session.

4. Intelligent Agent Integration

Each learner is paired with an adaptive agent that monitors their real-time mastery scores and engagement signals. When the agent detects a student struggling with a topic, it adjusts the difficulty of subsequent content, suggests remedial material, or triggers a real-time hint — all without waiting for the nightly batch analytics job, so personalisation happens within the same session the difficulty is detected.

5. Scalability & Distributed Computing Design

- Data partitioned by student-id so each learner's history stays co-located, minimising cross-node shuffling during agent queries.
- PySpark's distributed batch layer handles nightly cohort-wide model retraining while a lighter streaming layer handles per-session personalisation.
- New learners/courses scale horizontally by adding partitions and agent instances rather than redesigning the pipeline.

6. Data Flow & Processing Pipeline

Learning events flow from the LMS into distributed feature computation, into per-student mastery models, into the adaptive agent's recommendation decision, and finally into the content delivered to the learner, with session outcomes logged back into the analytics layer to refine future recommendations.

7. Innovation & Creativity

The innovation is session-level adaptivity at scale — most platforms personalise only between sessions via batch analytics, whereas pairing each learner with a lightweight agent operating on cached, distributed mastery profiles allows in-session adjustment for thousands of concurrent learners.